MARMARA ÜNİVERSİTESİ

Teknoloji Fakültesi / Bilgisayar Mühendisliği Bölümü

MANTIK DEVRELERİ — Ders 3 / İkili Aritmetik

Deney Yaprakları 1–4: Ek Dosya (kod, işlem kayıtları, test özetleri, gerekçeler)

Ad Soyad	
Öğrenci No	
Şube / Grup	
Tarih	

İçindekiler: Yaprak 1 — İkili toplayıcı · Yaprak 2 — Çıkarma · Yaprak 3 — Çarpıcı · Yaprak 4 — Bölme · Kısa cevaplar · Ek A — Kod

Deney Yaprağı 1 — Elde adımlarını gösteren ikili toplayıcı

1. Tek sütunun sonuç/elde tablosu ve algoritma

Bir sütunda iki bit (a, b) ve gelen elde c toplanır; 0–3 arası değer iki bitle (çıkan elde, sonuç) yazılır. Mantık ifadeleri: **S = a ⊕ b ⊕ c**, **C_{out} = a·b + c·(a ⊕ b)**.

a	b	c (gelen)	a+b+c	S (sonuç)	C_out (çıkan)
0	0	0	0	0	0
0	0	1	1	1	0
0	1	0	1	1	0
0	1	1	2	0	1
1	0	0	1	1	0
1	0	1	2	0	1
1	1	0	2	0	1
1	1	1	3	1	1

Algoritma (n bit, sağdan sola): c ← başlangıç eldesi; i = 0 … n−1 için (S_i, c) ← tam_toplayıcı(A_i, B_i, c) ve kayda (2^i, A_i, B_i, gelen c, S_i, çıkan c) yazılır; döngü sonundaki c son eldedir. Kod: ikili.py → tam_toplayici(), topla() (Ek A).

2. Dört işlem — programdan önce yazılan beklenen sonuçlar ve işlem kayıtları

İşlem	c0	Beklenen C S	Program: S	Son elde	Değer C·16+S	Uyum
0101 + 0011	0	0 1000	1000	0	8 = 5+3+0	✓
1111 + 0001	0	1 0000	0000	1	16 = 15+1+0	✓
1111 + 0000	1	1 0000	0000	1	16 = 15+0+1	✓
1111 + 1111	1	1 1111	1111	1	31 = 15+15+1	✓

0101 + 0011, başlangıç eldesi 0 → toplam 1000, son elde 0

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
A_i	0	1	0	1
B_i	0	0	1	1
gelen elde	1	1	1	0
sonuç S_i	1	0	0	0
çıkan elde	0	1	1	1

1111 + 0001, başlangıç eldesi 0 → toplam 0000, son elde 1

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
A_i	1	1	1	1
B_i	0	0	0	1
gelen elde	1	1	1	0
sonuç S_i	0	0	0	0
çıkan elde	1	1	1	1

1111 + 0000, başlangıç eldesi 1 → toplam 0000, son elde 1

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
A_i	1	1	1	1
B_i	0	0	0	0
gelen elde	1	1	1	1
sonuç S_i	0	0	0	0
çıkan elde	1	1	1	1

1111 + 1111, başlangıç eldesi 1 → toplam 1111, son elde 1

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
A_i	1	1	1	1
B_i	1	1	1	1
gelen elde	1	1	1	1
sonuç S_i	1	1	1	1
çıkan elde	1	1	1	1

3. 512 testlik kapsamlı sınama

Tüm A, B ∈ {0000 … 1111} ve c0 ∈ {0, 1}: 16 × 16 × 2 = **512** test. Her testte n bitlik çıktı ile son eldenin birlikte taşıdığı değer **C·2^4 + S** onlu referans **A + B + c0** (hazır aritmetik) ile karşılaştırıldı. Sonuç: **512 test, 0 hata**.

Ek denetim (aynı kod, farklı n): n=1: 8 test, 0 hata; n=2: 32 test, 0 hata; n=3: 128 test, 0 hata; n=4: 512 test, 0 hata; n=5: 2048 test, 0 hata; n=6: 8192 test, 0 hata; n=7: 32768 test, 0 hata; n=8: 131072 test, 0 hata.

4. Ret kuralları ve son eldenin anlamı

Kurallar: (i) A veya B boş dizgi ise ret; (ii) 0/1 dışında karakter içeriyorsa ret; (iii) A ve B uzunlukları farklı ise ret; (iv) başlangıç eldesi 0/1 değilse ret. Hepsi ValueError ile açık mesaj verir.

Durum	A	B	c0	Gözlenen
boş girdi	''	0101	0	ret: A: boş girdi kabul edilmez
geçersiz bit	'01a1'	0011	0	ret: A: geçersiz bit 'a' (yalnızca 0/1)
farklı uzunluk	'101'	0011	0	ret: farklı uzunluk: 3 ve 4 bit
geçersiz başlangıç eldesi	'0101'	0011	2	ret: başlangıç eldesi 0 ya da 1 olmalı

n bitlik toplayıcının gerçek sonucu **A + B + c0 = C·2^n + S**'dir; en büyük değer $(2^n-1)+(2^n-1)+1 = 2^{n+1}-1$ olduğundan n+1 bit gerekir ve (n+1). bit son eldedir. Son elde yok sayılırsa kalan yalnızca S = (A+B+c0) mod 2^n olur; kaybedilen değer **C·2^n**, yani C = 1 iken tam olarak **2^n** (n = 4 için 16).

İşlem	S	Son elde	Gerçek	Elde atılınca	Kayıp
1111+0001+0	0000	1	16	0	16 = 2^4
1111+1111+1	1111	1	31	15	16 = 2^4
1000+1000+0	0000	1	16	0	16 = 2^4

Sınır örneği: 1111 + 0001 = 1 0000 (16); elde atılınca 0000 = 0 kalır. Bu, iki pozitif sayının toplamının sıfır görünmesi demektir; işaretsiz taşma (overflow) göstergesi son eldedir.

Deney Yaprağı 2 — Borç alma ve 2 tümleyeniyle çıkarma

1. İki bağımsız algoritma

(a) Borç alarak doğrudan çıkarma (cikar_borc): sağdan sola her sütunda fark biti **D = m ⊕ n ⊕ b**, çıkan borç $\mathbf{B_{out} = m'\cdot n + (m \oplus n)'\cdot b}$. Son borç 0 ise M ≥ N ve sonuç fark dizisidir (fark 0000 ise sonuç “0”, işaretsiz). Son borç 1 ise M < N: işaret “−”, büyüklük aynı yöntemle N − M hesaplanarak bulunur.

(b) 2 tümleyeniyle çıkarma (cikar_tumleyen): M − N = M + N' + 1. N'nin bitleri terslenir, “+1” başlangıç eldesi olarak toplayıcıya girer (kendi tam toplayıcı zinciri; cikar_borc'u çağırmaz). Son elde 1 → M ≥ N, sonuç toplamın kendisi (0000 ise “0”). Son elde 0 → M < N, büyüklük = toplamın 2 tümleyeni (tersle + 1).

2. Beş işlemin iki yöntemle çözümü

İşlem	Onlu	Borç: son borç	Borç: sonuç	2 tüm.: son elde	2 tüm.: sonuç	Ref.
1110 − 0011	14 − 3	0	+1011	1	+1011	11
0011 − 1110	3 − 14	1	−1011	0	−1011	-11
1000 − 0001	8 − 1	0	+0111	1	+0111	7
0101 − 0101	5 − 5	0	0	1	0	0
0011 − 0000	3 − 0	0	+0011	1	+0011	3

1110 − 0011

(a) Borç zinciri M − N

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
M_i	1	1	1	0
N_i	0	0	1	1
gelen borç	0	1	1	0
fark D_i	1	0	1	1
çıkan borç	0	0	1	1

(b) N' = 1100; M + N' + 1 (başlangıç eldesi 1), son elde 1

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
M_i	1	1	1	0
N'_i	1	1	0	0
gelen elde	1	0	0	1
toplam	1	0	1	1
çıkan elde	1	1	0	0

Sonuç: **+1011** (onlu 11); iki yöntem aynı.

0011 − 1110

(a) Borç zinciri M − N

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
M_i	0	0	1	1
N_i	1	1	1	0
gelen borç	1	0	0	0
fark D_i	0	1	0	1
çıkan borç	1	1	0	0

(a) Son borç 1 → negatif; büyüklük için N − M

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
N_i	1	1	1	0
M_i	0	0	1	1
gelen borç	0	1	1	0
fark D_i	1	0	1	1
çıkan borç	0	0	1	1

(b) N' = 0001; M + N' + 1 (başlangıç eldesi 1), son elde 0

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
M_i	0	0	1	1
N'_i	0	0	0	1
gelen elde	0	1	1	1
toplam	0	1	0	1
çıkan elde	0	0	1	1

(b) Son elde 0 → negatif; büyüklük = toplamın 2 tümleyeni (terslenmiş toplam + 1)

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
tersl. toplam	1	0	1	0
0	0	0	0	0
gelen elde	0	0	0	1
büyüklük	1	0	1	1
çıkan elde	0	0	0	0

Sonuç: **−1011** (onlu -11); iki yöntem aynı.

1000 − 0001

(a) Borç zinciri M − N

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
M_i	1	0	0	0
N_i	0	0	0	1
gelen borç	1	1	1	0
fark D_i	0	1	1	1
çıkan borç	0	1	1	1

(b) N' = 1110; M + N' + 1 (başlangıç eldesi 1), son elde 1

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
M_i	1	0	0	0
N'_i	1	1	1	0
gelen elde	0	0	0	1
toplam	0	1	1	1
çıkan elde	1	0	0	0

Sonuç: **+0111** (onlu 7); iki yöntem aynı.

0101 − 0101

(a) Borç zinciri M − N

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
M_i	0	1	0	1
N_i	0	1	0	1
gelen borç	0	0	0	0
fark D_i	0	0	0	0
çıkan borç	0	0	0	0

(b) N' = 1010; M + N' + 1 (başlangıç eldesi 1), son elde 1

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
M_i	0	1	0	1
N'_i	1	0	1	0
gelen elde	1	1	1	1
toplam	0	0	0	0
çıkan elde	1	1	1	1

Sonuç: **0 (sıfır, işaretsiz)** (onlu 0); iki yöntem aynı.

0011 − 0000

(a) Borç zinciri M − N

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
M_i	0	0	1	1
N_i	0	0	0	0
gelen borç	0	0	0	0
fark D_i	0	0	1	1
çıkan borç	0	0	0	0

(b) N' = 1111; M + N' + 1 (başlangıç eldesi 1), son elde 1

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
M_i	0	0	1	1
N'_i	1	1	1	1
gelen elde	1	1	1	1
toplam	0	0	1	1
çıkan elde	1	1	1	1

Sonuç: **+0011** (onlu 3); iki yöntem aynı.

3. 256 operand çiftinin sınanması

M, N ∈ 0…15 için 16 × 16 = **256** çift. Her çiftte işaret+büyüklük onlu referans M − N ile ve sıfır durumunun “0” olarak verilmesi ayrıca karşılaştırıldı. Borç yöntemi: **0 hata**; 2 tümleyeni yöntemi: **0 hata**. Uyumsuz çift bulunmadı; düzeltme gerekmedi.

4. “Son elde yoksa sonuç mutlaka negatiftir” iddiası

N'nin 2 tümleyeni ayrı bir adımda 4 bitle hesaplanırsa: 0000 → tersle 1111 → +1 → 1 0000; sabit genişlikte taşan 1 atılır ve **2tüm(0000) = 0000** olur (sıfırın 2 tümleyeni yine sıfırdır). Sonra M + 0000 toplamında hiçbir zaman elde oluşmaz.

Bu “ayrı tümle, sonra eldesiz topla” sürümüyle 256 çift tarandı: iddia **16** çiftte yanlış karar verdi ve hepsinde N = 0000 (M = 0000 … 1111). **En küçük karşı örnek: 0000 − 0000** → toplam 0000, son elde 0, fakat gerçek sonuç 0 (negatif değil). Benzer biçimde 0011 − 0000: son elde yok, sonuç +3.

Karar kuralı: Çıkarma tek geçişte **M + N' + 1** olarak yapılmalı (“+1” başlangıç eldesi). N = 0 iken N' = 1111 ve 1111 + 1 her zaman son elde üretir; doğru yöntemde N = 0 için M=0000 → son elde 1, M=0011 → son elde 1, M=1111 → son elde 1. Böylece: **son elde 1 ⇔ M ≥ N** (sonuç ≥ 0; büyüklük toplamın kendisi, 0000 ise sıfır); **son elde 0 ⇔ M < N** (sonuç negatif; büyüklük toplamın 2 tümleyeni). 2 tümleyeni ayrı hesaplanacaksa N = 0 özel durumu (sonuç = M, işaret +/0) ayrıca ele alınmalıdır.

Deney Yaprağı 3 — Kısmi çarpımlarla ikili çarpıcı

1. Kısmi çarpım ve kaydırma kuralı

Çarpan B'nin i konumundaki biti b_i (sağdan 0'dan başlayarak) için kısmi çarpım **$PP_i = (b_i \cdot A) \ll i$**: $b_i = 1$ ise A, $b_i = 0$ ise n tane 0 alınır; sağına i adet 0 eklenerek i kez sola kaydırılır ve soldan sıfırla n+m bite tamamlanır (başlangıç sıfırları korunur). Çarpım **$P = \Sigma\ PP_i$**; toplama, Yaprak 1'deki bit düzeyindeki topla() ile n+m bitlik biriktiriciye yapılır. Ana sonuçta hazır çarpma ya da tam sayıya dönüşüm kullanılmaz (kismi_carpim(), carp(), Ek A).

2. Hizalı kısmi çarpım tabloları

1011 × 101 (n = 4, m = 3, çıktı 7 bit): P = **0110111**. Bağımsız denetim: 11 × 5 = 55 ✓

i	b_i	PP_i (7 bit)	Onlu katkı
0	1	0001011	1·11·2^0 = 11
1	0	0000000	0·11·2^1 = 0
2	1	0101100	1·11·2^2 = 44
	Σ	0110111	**11 + 0 + 44 = 55**

10111 × 110 (n = 5, m = 3, çıktı 8 bit): P = **10001010**. Bağımsız denetim: 23 × 6 = 138 ✓

i	b_i	PP_i (8 bit)	Onlu katkı
0	0	00000000	0·23·2^0 = 0
1	1	00101110	1·23·2^1 = 46
2	1	01011100	1·23·2^2 = 92
	Σ	10001010	**0 + 46 + 92 = 138**

11001 × 1010 (n = 5, m = 4, çıktı 9 bit): P = **011111010**. Bağımsız denetim: 25 × 10 = 250 ✓

i	b_i	PP_i (9 bit)	Onlu katkı
0	0	000000000	0·25·2^0 = 0
1	1	000110010	1·25·2^1 = 50
2	0	000000000	0·25·2^2 = 0
3	1	011001000	1·25·2^3 = 200
	Σ	011111010	**0 + 50 + 0 + 200 = 250**

3. Testler ve sıfır kısmi çarpımı atlayan ikinci sürüm

Sayım kuralı: biriktirici 0…0 ile başlar; biriktiriciye eklenen her kısmi çarpım bir toplama çağrısıdır. Sürüm 1 her biti işler → **m** çağrı. Sürüm 2 $b_i = 0$ olan satırları atlar → **B'deki 1 sayısı** kadar çağrı (çarpan 0 ise hiç çağrı yok).

Test	A	B	Sürüm 1	Sürüm 2	Çağrı v1	Çağrı v2	Ref.	Uyum
çarpan sıfır	1011	0	00000	00000	1	0	0	✓
çarpan bir	1011	1	01011	01011	1	1	11	✓
çarpan 2^2	1011	100	0101100	0101100	3	1	44	✓
çarpan 2^3	101	1000	0101000	0101000	4	1	40	✓
çarpan tüm 1	1011	111	1001101	1001101	3	3	77	✓
iki operand tüm 1	1111	1111	11100001	11100001	4	4	225	✓
çarpılan sıfır	0	101	0000	0000	3	2	0	✓
1 x 1	1	1	01	01	1	1	1	✓
n+m-1 karşı örneği	11	11	1001	1001	2	2	9	✓
seyrek çarpan	110	10001	01100110	01100110	5	2	102	✓

Ek tarama: n, m = 1…6 bitlik tüm operand çiftleri (15876 çarpım) için iki sürüm hem birbiriyle hem de onlu referansla karşılaştırıldı: **0 hata**. İki sürüm her zaman aynı sonucu verir; fark yalnızca toplama sayısındadır (sıfır satır eklemek sonucu değiştirmez).

4. Çıktı genişliği gerekçesi

n bitlik en büyük işaretsiz sayı 2^n-1, m bitlik 2^m-1'dir. Üst sınır: $(2^n-1)(2^m-1) = 2^{n+m} - 2^n - 2^m + 1 < 2^{n+m}$ (n, m ≥ 1 için $2^n + 2^m - 1 > 0$). Dolayısıyla her çarpım n+m bite sığar; örn. 1111 × 1111 = 225 = 11100001 (8 bit).

n+m−1 bit her zaman yetmez: **11 × 11 = 3 × 3 = 9 = 1001**; n+m−1 = 3 bit en çok 7'yi gösterir, 9 için 4 bit gerekir. (n veya m = 1 iken çarpım ≤ $2^{n}-1$ olduğundan n+m−1 bit yeter; bu yüzden en küçük karşı örnek n = m = 2'dir.) Öte yandan en soldaki bitleri 1 olan operandlarda çarpım ≥ $2^{n-1}\cdot 2^{m-1} = 2^{n+m-2}$ olduğundan en az n+m−1 bit her zaman gerekir.

Deney Yaprağı 4 — Bölüm, kalan ve kesirli bölme

1. Soldan sağa bölme algoritması

Kalan r = 0 ile başlanır. Bölünenin her biti için (soldan sağa): incelenen parça = r'nin sağına sıradaki bitin indirilmesi (r·2 + bit). Parça ≥ d ise bölüm biti 1 ve r = parça − d (bit düzeyinde borçlu çıkarma), değilse bölüm biti 0 ve r = parça. Karşılaştırma bit düzeyindedir (önce anlamlı uzunluk, sonra soldan ilk farklı bit). Bölen yalnızca sıfırlardan oluşuyorsa ZeroDivisionError ile açıkça reddedilir. Hazır bölme/kalan operatörü kullanılmaz (buyuk_esit(), bol(), Ek A).

2–3. Üç işlem: tam bölüm, kalan ve kesirli devam

10110 ÷ 100 (22 ÷ 4)

Adım	İncelenen parça	Bölüm biti	Kalan
1	1	0	1
2	10	0	10
3	101	1	1
4	11	0	11
5	110	1	10

Q = **101** (5), R = **10** (2). Kontrol 1: Q·d + R = 5·4 + 2 = 22 = D ✓; Kontrol 2: 0 ≤ R = 2 < d = 4 ✓. Referans (//, %): (5, 2).

Kesir adımı	İncelenen parça	Bölüm biti	Kalan
1	100	1	0

Kesirli sonuç: 10110 ÷ 100 = **101.1**$_2$ (onlu = 5.5; gerçek 5.5).

1101 ÷ 10 (13 ÷ 2)

Adım	İncelenen parça	Bölüm biti	Kalan
1	1	0	1
2	11	1	1
3	10	1	0
4	1	0	1

Q = **110** (6), R = **1** (1). Kontrol 1: Q·d + R = 6·2 + 1 = 13 = D ✓; Kontrol 2: 0 ≤ R = 1 < d = 2 ✓. Referans (//, %): (6, 1).

Kesir adımı	İncelenen parça	Bölüm biti	Kalan
1	10	1	0

Kesirli sonuç: 1101 ÷ 10 = **110.1**$_2$ (onlu = 6.5; gerçek 6.5).

111 ÷ 11 (7 ÷ 3)

Adım	İncelenen parça	Bölüm biti	Kalan
1	1	0	1
2	11	1	0
3	1	0	1

Q = **10** (2), R = **1** (1). Kontrol 1: Q·d + R = 2·3 + 1 = 7 = D ✓; Kontrol 2: 0 ≤ R = 1 < d = 3 ✓. Referans (//, %): (2, 1).

Kesir adımı	İncelenen parça	Bölüm biti	Kalan
1	10	0	10
2	100	1	1
3	10	0	10
4	100	1	1
5	10	0	10
6	100	1	1
7	10	0	10
8	100	1	1

Kesirli sonuç: 111 ÷ 11 ≈ **10.01010101**$_2$ (onlu ≈ 2.3320312; gerçek 2.3333333). Kalan 1 (1) kesir adımı 0'dan sonra adım 2'de yeniden görüldü → basamaklar “01” periyoduyla tekrar eder; kesir hiç bitmez, 8 basamakta kesildi.

İki koşul neden birlikte gerekli?

Yalnızca D = Q·d + R yetmez: 10110 ÷ 100 için Q = 100 (4), R = 110 (6) de 4·4 + 6 = 22 sağlar, ama R ≥ d olduğundan bölüm eksiktir (Q = 0, R = D de her zaman sağlar). Yalnızca 0 ≤ R < d de yetmez: Q = 100, R = 10 bu koşulu sağlar fakat 4·4 + 2 = 18 ≠ 22. İki koşul birlikte (Q, R) çiftini **tek** olarak belirler (bölme teoremi).

Durma koşulları (111 ÷ 11 üzerinden)

Tam bölme bölünenin bit sayısı kadar adımda kesin olarak durur (111 için 3 adım): indirilecek bit kalmadığında Q = 10, R = 1 kesinleşir. **Kesirli bölme** ise kalandan devam eder (kalanın sağına 0 indirilir); (a) kalan 0 olursa sonuç tamamlanmıştır ve “=” yazılır, (b) 8 basamağa ulaşılırsa kesilir ve “≈” yazılır. 111 ÷ 11'de kalanlar 1 → 10 → 1 → 10 … biçiminde tekrar eder; bir kalan yeniden görüldüğünde kesir periyodiktir ve asla 0'a ulaşmaz: 7/3 = 10.0101…$_2$ ≈ 10.01010101$_2$. 10110 ÷ 100 ve 1101 ÷ 10 ise 1 kesir basamağında kalan 0 olduğundan “=” ile tamamlanır (ek sıfır üretilmedi).

4. Sınır testleri

Durum	D	d	Beklenen Q, R	Gözlenen Q, R	Uyum
sıfır bölünen	0	101	0, 0	0, 0	✓
sıfır bölünen (3 bit)	000	11	0, 0	0, 0	✓
bölen > bölünen	11	101	0, 11	0, 11	✓
eşit operandlar	101	101	1, 0	1, 0	✓
eşit operandlar	1011	1011	1, 0	1, 0	✓
sıfır bölen	1010	000	ret	ret: bölen sıfır: bölme tanımsız, girdi reddedildi	✓

Bölümde sıfır bitinin yanlış atılması: bol_hatali() bölen sığmadığında bölüme 0 yazmayı atlar. Bu hata baştaki sıfırlarda zararsızdır, ama ara sıfırlarda bölümü bozar:

D ÷ d	Doğru Q	Hatalı Q	Hatalı R	Q·d + R = D ?
10110 ÷ 100	101	11	10	3·4+2 = 14 ≠ 22 → YAKALANDI
1101 ÷ 10	110	11	1	3·2+1 = 7 ≠ 13 → YAKALANDI
100100 ÷ 11	1100	11	0	3·3+0 = 9 ≠ 36 → YAKALANDI

Örn. 10110 ÷ 100: doğru bölüm 101'dir; ortadaki 0 atılınca 11 (3) çıkar ve 3·4 + 2 = 14 ≠ 22 olduğundan D = Q·d + R kontrolü hatayı yakalar (kalan doğru olduğu için R < d kontrolü tek başına yakalayamaz — iki kontrolün birlikte gerekliliğine bir örnek daha).

Ek tarama: D 1–8 bit, d = 1…15 tüm kombinasyonlar (7650 bölme) // ve % ile karşılaştırıldı: **0 hata**.

Kısa cevap ve doğrulama notları (deney yapraklarına yazılanlar)

Deney Yaprağı 1

İlk tahminim	0101+0011 = 1000 (C=0); 1111+0001 = 0000 (C=1); 1111+0000+1 = 0000 (C=1); 1111+1111+1 = 1111 (C=1). Gerçek değer = $C \cdot 2^n + S$.
Bağımsız kontrolüm	$16 \cdot 16 \cdot 2 = 512$ testin hepsinde $C \cdot 16 + S$ ile onlu A+B+c0 karşılaştırıldı: 0 hata. Aynı kod n = 1…8 için de 0 hata verdi.
Karşı örneğim	Son elde atılırsa 1111+0001 → 0000 (16 yerine 0); kayıp $C \cdot 2^n = 2^4 = 16$. 1111+1111+1 = 31 → 1111 = 15 kalır.
YZ beyanı	Kod/tablo taslağında YZ (Claude) kullandım; sonuçları tam tarama testi ve elle hesapla doğruladım. 1111+1111+1 = 11111 elle denetlendi.

Deney Yaprağı 2

İlk tahminim	1110−0011 = +1011 (11); 0011−1110 = −1011 (−11); 1000−0001 = +0111 (7); 0101−0101 = 0; 0011−0000 = +0011 (3).
Bağımsız kontrolüm	256 çiftte borç ve 2 tümleyeni yöntemleri ayrı ayrı onlu M−N ile karşılaştırıldı: ikisinde de 0 hata, uyumsuz çift yok.
Karşı örneğim	0000−0000: 2tüm(0000) = 0000 (taşan 1 atılır), 0000+0000 son elde 0 ama sonuç 0, negatif değil. Doğru kural: M+N'+1 tek geçişte; elde 1 ⇔ M ≥ N.
YZ beyanı	Kod/tablo taslağında YZ (Claude) kullandım; sonuçları tam tarama testi ve elle hesapla doğruladım. N = 0 durumu ayrıca tarandı.

Deney Yaprağı 3

İlk tahminim	1011×101 = 0110111 (55); 10111×110 = 10001010 (138); 11001×1010 = 011111010 (250).
Bağımsız kontrolüm	Onlu katkılar 11+0+44 = 55, 0+46+92 = 138, 0+50+0+200 = 250; 10 test ve 15876 çarpımlık tarama 0 hata. Toplama: v1 = m, v2 = B'deki 1 sayısı.
Karşı örneğim	$(2^n-1)(2^m-1) < 2^{n+m}$ → n+m bit yeter. 11×11 = 1001 (9) 4 bit ister; n+m−1 = 3 bit en çok 7 gösterir.
YZ beyanı	Kod/tablo taslağında YZ (Claude) kullandım; sonuçları tam tarama testi ve elle hesapla doğruladım. Kısmi çarpımlar elle toplandı.

Deney Yaprağı 4

İlk tahminim	10110÷100: Q = 101, R = 10 (= 101.1); 1101÷10: Q = 110, R = 1 (= 110.1); 111÷11: Q = 10, R = 1 (≈ 10.01010101).
Bağımsız kontrolüm	Her işlemde Q·d+R = D ve 0 ≤ R < d ayrı ayrı sağlandı; // ve % ile 7650 bölmelik tarama 0 hata. 111÷11'de kalanlar 1, 10, 1, 10... tekrar eder.
Karşı örneğim	Ara sıfırı atan sürüm 10110÷100 için Q = 11 verir: 11·100+10 = 1110 (14) ≠ 10110 (22). Q = 100, R = 110 de Q·d+R = D sağlar ama R ≥ d.
YZ beyanı	Kod/tablo taslağında YZ (Claude) kullandım; sonuçları tam tarama testi ve elle hesapla doğruladım. Sıfır bölen reddi sınandı.

Ek A — Kod (Python 3)

Çalıştırma: `python3 testler.py` (tüm testler), `python3 rapor_olustur.py` (bu dosya).

ikili.py

```
"""Mantık Devreleri - Ders 3 / İkili Aritmetik deney yaprakları 1-4.

Ana hesaplar yalnızca bit düzeyinde (karakter '0'/'1' ve mantık işlemleri) yapılır.
Hazır aritmetik (int(), +, -, *, //, %) yalnızca test referanslarında ve
onlu gösterimlerde kullanılır.
"""

# ---------------------------------------------------------------------------
# Ortak yardımcılar
# ---------------------------------------------------------------------------

def dogrula(bitler, ad="girdi"):
    """Boş girdi ve geçersiz bit için ret kuralı."""
    if not isinstance(bitler, str) or len(bitler) == 0:
        raise ValueError(f"{ad}: boş girdi kabul edilmez")
    for ch in bitler:
        if ch not in "01":
            raise ValueError(f"{ad}: geçersiz bit '{ch}' (yalnızca 0/1)")


def tersle(bitler):
    """Her biti tersler (1 tümleyeni)."""
    return "".join("1" if b == "0" else "0" for b in bitler)


def sola_doldur(bitler, genislik):
    return "0" * (genislik - len(bitler)) + bitler


def bas_sifir_at(bitler):
    i = 0
    while i < len(bitler) - 1 and bitler[i] == "0":
        i += 1
    return bitler[i:]


# ---------------------------------------------------------------------------
# Yaprak 1 - Elde adımlarını gösteren ikili toplayıcı
# ---------------------------------------------------------------------------

def tam_toplayici(a, b, c):
    """Bir sütun: (sonuç biti, çıkan elde). a, b, c in {0, 1}."""
    s = a ^ b ^ c
    c_out = (a & b) | (c & (a ^ b))
    return s, c_out


def dogruluk_tablosu():
    satirlar = []
    for a in (0, 1):
        for b in (0, 1):
            for c in (0, 1):
                s, co = tam_toplayici(a, b, c)
                satirlar.append((a, b, c, s, co))
    return satirlar


def topla(A, B, elde0=0):
    """n bitlik A + B + elde0. Döner: (toplam, son_elde, sütun_kaydı).

    sütun_kaydı satırı: (ağırlık i, a_i, b_i, gelen elde, sonuç biti, çıkan elde)
    """
    dogrula(A, "A")
    dogrula(B, "B")
    if len(A) != len(B):
        raise ValueError(f"farklı uzunluk: {len(A)} ve {len(B)} bit")
    if elde0 not in (0, 1):
        raise ValueError("başlangıç eldesi 0 ya da 1 olmalı")
    n = len(A)
    c = elde0
    sonuc = [""] * n
    kayit = []
    for i in range(n):              # sağdan sola: ağırlık 2^i
        a = 1 if A[n - 1 - i] == "1" else 0
        b = 1 if B[n - 1 - i] == "1" else 0
        s, co = tam_toplayici(a, b, c)
        kayit.append((i, a, b, c, s, co))
        sonuc[n - 1 - i] = "1" if s else "0"
        c = co
    return "".join(sonuc), c, kayit
# ---------------------------------------------------------------------------
```

```
# Yaprak 2 - Borç alma ve 2 tümleyeniyle çıkarma
# ------------------------------------------------------------------------

def _borcla_cikar_ham(M, N):
    """Sabit genişlikte M - N, borç alarak. Döner: (fark, son_borç, kayıt)."""
    n = len(M)
    borc = 0
    fark = [""] * n
    kayit = []
    for i in range(n):
        m = 1 if M[n - 1 - i] == "1" else 0
        k = 1 if N[n - 1 - i] == "1" else 0
        d = m ^ k ^ borc
        b_out = ((1 - m) & k) | ((1 - (m ^ k)) & borc)
        kayit.append((i, m, k, borc, d, b_out))
        fark[n - 1 - i] = "1" if d else "0"
        borc = b_out
    return "".join(fark), borc, kayit


def cikar_borc(M, N):
    """Doğrudan çıkarma. Döner: (işaret, büyüklük, kayıtlar).

    Son borç 1 ise M < N'dir; büyüklük aynı yöntemle N - M hesaplanarak bulunur.
    """
    dogrula(M, "M")
    dogrula(N, "N")
    if len(M) != len(N):
        raise ValueError("farklı uzunluk")
    fark, borc, kayit = _borcla_cikar_ham(M, N)
    kayitlar = {"M-N": kayit, "son_borc": borc}
    if borc == 0:
        # sıfır durumu ayrıca: işaretsiz "0"
        return ("+" if "1" in fark else "0"), fark, kayitlar
    buyukluk, _, kayit2 = _borcla_cikar_ham(N, M)
    kayitlar["N-M"] = kayit2
    return "-", buyukluk, kayitlar


def _tumleyen_topla_ham(X, Y, elde0):
    """2 tümleyeni yöntemi için kendi tam toplayıcı zinciri (cikar_borc'u çağırmaz)."""
    n = len(X)
    c = elde0
    s = [""] * n
    kayit = []
    for i in range(n):
        a = 1 if X[n - 1 - i] == "1" else 0
        b = 1 if Y[n - 1 - i] == "1" else 0
        t, co = tam_toplayici(a, b, c)
        kayit.append((i, a, b, c, t, co))
        s[n - 1 - i] = "1" if t else "0"
        c = co
    return "".join(s), c, kayit


def cikar_tumleyen(M, N):
    """M - N = M + (N'nin 1 tümleyeni) + 1. '+1' başlangıç eldesi olarak girer.

    Son elde 1 -> M >= N, sonuç doğrudan toplamdır.
    Son elde 0 -> M < N, büyüklük = toplamın 2 tümleyeni.
    """
    dogrula(M, "M")
    dogrula(N, "N")
    if len(M) != len(N):
        raise ValueError("farklı uzunluk")
    n = len(M)
    N1 = tersle(N)
    toplam, son_elde, kayit = _tumleyen_topla_ham(M, N1, 1)
    kayitlar = {"N_1tum": N1, "M+N'+1": kayit, "son_elde": son_elde}
    if son_elde == 1:
        if "1" in toplam:
            return "+", toplam, kayitlar
        return "0", toplam, kayitlar
    T1 = tersle(toplam)
    buyukluk, _, kayit2 = _tumleyen_topla_ham(T1, "0" * n, 1)
    kayitlar["buyukluk"] = kayit2
    return "-", buyukluk, kayitlar


def iki_tumleyeni(N):
    """Sabit genişlikte 2 tümleyeni (taşan elde atılır). Döner: (sonuç, atılan elde)."""
    s, c, _ = _tumleyen_topla_ham(tersle(N), "0" * len(N), 1)
    return s, c


def cikar_tumleyen_hatali(M, N):
    """Hatalı yaklaşım: önce 2 tümleyenini ayrı hesaplayıp kesmek, sonra eldesiz toplamak.
    'Son elde yoksa negatif' kuralı burada N = 0 için yanlış karar verir."""
```

```
    N2, _ = iki_tumleyeni(N)
    toplam, son_elde, _ = _tumleyen_topla_ham(M, N2, 0)
    return ("+" if son_elde == 1 else "-"), toplam, son_elde


# ---------------------------------------------------------------------------
# Yaprak 3 - Kısmi çarpımlarla ikili çarpıcı
# ---------------------------------------------------------------------------

def kismi_carpim(A, b_i, i, genislik):
    """i konumundaki çarpan biti için: b_i = 1 ise A, değilse 0...0;
    sağa i adet 0 eklenerek i kez sola kaydırılır, n+m bite sola sıfırla doldurulur."""
    taban = A if b_i == "1" else "0" * len(A)
    return sola_doldur(taban + "0" * i, genislik)


def carp(A, B, sifirlari_atla=False):
    """A (n bit) x B (m bit) -> n+m bit. Döner: (çarpım, kısmi_çarpımlar, toplama_sayısı).

    Sayım kuralı: biriktirici 0...0 ile başlar; biriktiriciye eklenen her kısmi
    çarpım bir toplama çağrısıdır. v1: m çağrı, v2 (sıfırları atla): B'deki 1 sayısı.
    """
    dogrula(A, "çarpılan")
    dogrula(B, "çarpan")
    n, m = len(A), len(B)
    g = n + m
    acc = "0" * g
    kismiler = []
    cagri = 0
    for i in range(m):
        b_i = B[m - 1 - i]
        pp = kismi_carpim(A, b_i, i, g)
        kismiler.append((i, b_i, pp))
        if sifirlari_atla and b_i == "0":
            continue
        acc, son_elde, _ = topla(acc, pp, 0)
        cagri += 1
        assert son_elde == 0, "n+m bit yetmedi (olmamalı)"
    return acc, kismiler, cagri


# ---------------------------------------------------------------------------
# Yaprak 4 - Bölüm, kalan ve kesirli bölme
# ---------------------------------------------------------------------------

def buyuk_esit(X, Y):
    """X >= Y ? (baştaki sıfırlar atılarak; önce uzunluk, sonra soldan ilk farklı bit)."""
    X, Y = bas_sifir_at(X), bas_sifir_at(Y)
    if len(X) != len(Y):
        return len(X) > len(Y)
    for x, y in zip(X, Y):
        if x != y:
            return x == "1"
    return True


def _fark(X, Y):
    """X >= Y varsayımıyla X - Y (borçlu çıkarma), baştaki sıfırlar atılmış."""
    g = max(len(X), len(Y))
    f, borc, _ = _borcla_cikar_ham(sola_doldur(X, g), sola_doldur(Y, g))
    assert borc == 0
    return bas_sifir_at(f)


def bol(D, d, kesir_basamak=0):
    """Soldan sağa uzun bölme. Döner: sözlük (Q, R, adimlar, kesir, kesir_adim, tam_mi)."""
    dogrula(D, "bölünen")
    dogrula(d, "bölen")
    if "1" not in d:
        raise ZeroDivisionError("bölen sıfır: bölme tanımsız, girdi reddedildi")
    if not (0 <= kesir_basamak <= 8):
        raise ValueError("kesir basamağı 0..8 olmalı")
    r = "0"
    q = []
    adimlar = []
    for j, bit in enumerate(D):
        parca = bas_sifir_at(r + bit)          # kalanı sola kaydır, sıradaki biti indir
        if buyuk_esit(parca, d):
            qb = "1"
            r = _fark(parca, d)
        else:
            qb = "0"
            r = parca
        q.append(qb)
        adimlar.append((j + 1, parca, qb, r))
    Q = bas_sifir_at("".join(q))
    R = r
    kesir = []
```

```
    kesir_adim = []
    gorulen = {R: 0}
    tekrar = None
    while R != "0" and len(kesir) < kesir_basamak:
        parca = bas_sifir_at(R + "0")
        if buyuk_esit(parca, d):
            fb = "1"
            R = _fark(parca, d)
        else:
            fb = "0"
            R = parca
        kesir.append(fb)
        kesir_adim.append((len(kesir), parca, fb, R))
        if R in gorulen and tekrar is None and R != "0":
            tekrar = (gorulen[R], len(kesir), R)
        gorulen.setdefault(R, len(kesir))
    return {
        "Q": Q, "R": r, "adimlar": adimlar,
        "kesir": "".join(kesir), "kesir_adim": kesir_adim,
        "tam_mi": R == "0", "tekrar": tekrar,
    }


def bol_hatali(D, d):
    """Hatalı sürüm: bölen sığmadığında bölüme 0 yazmayı atlar (sıfır biti düşer)."""
    r = "0"
    q = []
    for bit in D:
        parca = bas_sifir_at(r + bit)
        if buyuk_esit(parca, d):
            q.append("1")
            r = _fark(parca, d)
        else:
            r = parca                       # HATA: q.append("0") unutuldu
    return bas_sifir_at("".join(q) or "0"), r
```

testler.py

```
"""Deney 3 testleri. Referans değerler hazır aritmetikle (int, +, -, *, //, %) hesaplanır.

Çalıştırma:  python3 deney03/testler.py
"""
from itertools import product

from ikili import (topla, dogruluk_tablosu, cikar_borc, cikar_tumleyen,
                   cikar_tumleyen_hatali, iki_tumleyeni, carp, bol, bol_hatali)


def ikili(x, n):
    return format(x, f"0{n}b")


def onlu_isaretli(isaret, buyukluk):
    v = int(buyukluk, 2)
    return -v if isaret == "-" else v


# ------------------------------ Yaprak 1 ---------------------------------

def yaprak1():
    sonuc = {"tablo": dogruluk_tablosu()}
    islemler = [("0101", "0011", 0, "1000", 0), ("1111", "0001", 0, "0000", 1),
                ("1111", "0000", 1, "0000", 1), ("1111", "1111", 1, "1111", 1)]
    kayitlar = []
    for A, B, c0, beklenen_s, beklenen_c in islemler:
        s, c, kayit = topla(A, B, c0)
        kayitlar.append({"A": A, "B": B, "c0": c0, "S": s, "C": c, "kayit": kayit,
                         "beklenen": (beklenen_s, beklenen_c),
                         "uyum": (s, c) == (beklenen_s, beklenen_c)})
    sonuc["islemler"] = kayitlar

    n = 4
    toplam_test = hata = 0
    for a, b, c0 in product(range(16), range(16), (0, 1)):
        s, c, _ = topla(ikili(a, n), ikili(b, n), c0)
        toplam_test += 1
        if c * 2 ** n + int(s, 2) != a + b + c0:
            hata += 1
    sonuc["kapsamli"] = (toplam_test, hata)

    # genelleme: n = 1..8 için de aynı kontrol (n=8: 2*256*256 test)
    genel = []
    for n2 in range(1, 9):
        h = 0
        t = 0
        for a, b, c0 in product(range(2 ** n2), range(2 ** n2), (0, 1)):
            s, c, _ = topla(ikili(a, n2), ikili(b, n2), c0)
            t += 1
```

```
            h += c * 2 ** n2 + int(s, 2) != a + b + c0
        genel.append((n2, t, h))
    sonuc["genel"] = genel

    retler = []
    for A, B, c0, aciklama in [("", "0101", 0, "boş girdi"),
                               ("01a1", "0011", 0, "geçersiz bit"),
                               ("101", "0011", 0, "farklı uzunluk"),
                               ("0101", "0011", 2, "geçersiz başlangıç eldesi")]:
        try:
            topla(A, B, c0)
            retler.append((aciklama, A, B, c0, "KABUL EDİLDİ (HATA)"))
        except ValueError as e:
            retler.append((aciklama, A, B, c0, f"ret: {e}"))
    sonuc["retler"] = retler

    # son eldeyi yok sayma: kayıp = c * 2^n ; en büyük kayıp örneği
    kayip = []
    for A, B, c0 in [("1111", "0001", 0), ("1111", "1111", 1), ("1000", "1000", 0)]:
        s, c, _ = topla(A, B, c0)
        gercek = int(A, 2) + int(B, 2) + c0
        kayip.append((A, B, c0, s, c, gercek, int(s, 2), gercek - int(s, 2)))
    sonuc["kayip"] = kayip
    return sonuc


# ------------------------------ Yaprak 2 ---------------------------------

def yaprak2():
    sonuc = {}
    islemler = ["1110-0011", "0011-1110", "1000-0001", "0101-0101", "0011-0000"]
    kayitlar = []
    for s in islemler:
        M, N = s.split("-")
        i1, b1, k1 = cikar_borc(M, N)
        i2, b2, k2 = cikar_tumleyen(M, N)
        kayitlar.append({"M": M, "N": N, "borc": (i1, b1, k1), "tum": (i2, b2, k2),
                         "ref": int(M, 2) - int(N, 2)})
    sonuc["islemler"] = kayitlar

    test = hata1 = hata2 = 0
    ilk_hata = None
    for m, k in product(range(16), range(16)):
        M, N = ikili(m, 4), ikili(k, 4)
        ref = m - k
        i1, b1, _ = cikar_borc(M, N)
        i2, b2, _ = cikar_tumleyen(M, N)
        test += 1
        ok1 = onlu_isaretli(i1, b1) == ref and (i1 == "0") == (ref == 0)
        ok2 = onlu_isaretli(i2, b2) == ref and (i2 == "0") == (ref == 0)
        hata1 += not ok1
        hata2 += not ok2
        if (not ok1 or not ok2) and ilk_hata is None:
            ilk_hata = (M, N)
    sonuc["kapsamli"] = (test, hata1, hata2, ilk_hata)

    # "Son elde yoksa sonuç mutlaka negatiftir" iddiası
    yanlis = []
    for m, k in product(range(16), range(16)):
        M, N = ikili(m, 4), ikili(k, 4)
        isaret, toplam, c = cikar_tumleyen_hatali(M, N)
        if (isaret == "-") != (m - k < 0):
            yanlis.append((M, N, toplam, c))
    sonuc["iddia_yanlis"] = yanlis
    sonuc["sifir_tumleyen"] = iki_tumleyeni("0000")
    # doğru yöntemde (M + N' + 1) N = 0 için son elde
    sonuc["dogru_n0"] = [(ikili(m, 4), cikar_tumleyen(ikili(m, 4), "0000")[2]["son_elde"])
                         for m in (0, 3, 15)]
    return sonuc


# ------------------------------ Yaprak 3 ---------------------------------

def yaprak3():
    sonuc = {}
    tablolar = []
    for A, B in [("1011", "101"), ("10111", "110"), ("11001", "1010")]:
        P, kismiler, cagri = carp(A, B)
        tablolar.append({"A": A, "B": B, "P": P, "kismiler": kismiler,
                         "katkilar": [int(pp, 2) for _, _, pp in kismiler],
                         "ref": int(A, 2) * int(B, 2)})
    sonuc["tablolar"] = tablolar

    testler = [("1011", "0", "çarpan sıfır"), ("1011", "1", "çarpan bir"),
               ("1011", "100", "çarpan 2^2"), ("101", "1000", "çarpan 2^3"),
               ("1011", "111", "çarpan tüm 1"), ("1111", "1111", "iki operand tüm 1"),
               ("0", "101", "çarpılan sıfır"), ("1", "1", "1 x 1"),
               ("11", "11", "n+m-1 karşı örneği"), ("110", "10001", "seyrek çarpan")]
```

```
    satirlar = []
    for A, B, ad in testler:
        P1, _, c1 = carp(A, B)
        P2, _, c2 = carp(A, B, sifirlari_atla=True)
        ref = int(A, 2) * int(B, 2)
        satirlar.append((ad, A, B, P1, P2, c1, c2, ref,
                         int(P1, 2) == ref and P1 == P2 and len(P1) == len(A) + len(B)
                         and c2 == B.count("1")))
    sonuc["testler"] = satirlar

    # ek: tüm 1..6 x 1..6 bit kombinasyonları
    t = h = 0
    for n in range(1, 7):
        for m in range(1, 7):
            for a in range(2 ** n):
                for b in range(2 ** m):
                    A, B = ikili(a, n), ikili(b, m)
                    P1, _, _ = carp(A, B)
                    P2, _, _ = carp(A, B, True)
                    t += 1
                    h += not (int(P1, 2) == a * b and P1 == P2)
    sonuc["kapsamli"] = (t, h)
    return sonuc


# ------------------------------ Yaprak 4 ---------------------------------

def yaprak4():
    sonuc = {}
    islemler = []
    for D, d in [("10110", "100"), ("1101", "10"), ("111", "11")]:
        r = bol(D, d, kesir_basamak=8)
        Dv, dv, Qv, Rv = int(D, 2), int(d, 2), int(r["Q"], 2), int(r["R"], 2)
        r.update(D=D, d=d, kontrol1=(Qv * dv + Rv == Dv), kontrol2=(0 <= Rv < dv),
                 ref=(Dv // dv, Dv % dv), deger=Dv / dv)
        islemler.append(r)
    sonuc["islemler"] = islemler

    sinir = []
    for D, d, ad, beklenen in [("0", "101", "sıfır bölünen", ("0", "0")),
                               ("000", "11", "sıfır bölünen (3 bit)", ("0", "0")),
                               ("11", "101", "bölen > bölünen", ("0", "11")),
                               ("101", "101", "eşit operandlar", ("1", "0")),
                               ("1011", "1011", "eşit operandlar", ("1", "0"))]:
        r = bol(D, d)
        sinir.append((ad, D, d, beklenen, (r["Q"], r["R"]), (r["Q"], r["R"]) == beklenen))
    try:
        bol("1010", "000")
        sinir.append(("sıfır bölen", "1010", "000", "ret", "KABUL EDİLDİ", False))
    except ZeroDivisionError as e:
        sinir.append(("sıfır bölen", "1010", "000", "ret", f"ret: {e}", True))
    sonuc["sinir"] = sinir

    hatali = []
    for D, d in [("10110", "100"), ("1101", "10"), ("100100", "11")]:
        Qh, Rh = bol_hatali(D, d)
        Q = bol(D, d)["Q"]
        hatali.append((D, d, Q, Qh, Rh,
                       int(Qh, 2) * int(d, 2) + int(Rh, 2) == int(D, 2)))
    sonuc["hatali"] = hatali

    # ek: D 1..8 bit, d 1..15 tüm kombinasyonlar
    t = h = 0
    for nb in range(1, 9):
        for Dv in range(2 ** nb):
            for dv in range(1, 16):
                r = bol(ikili(Dv, nb), format(dv, "b"))
                t += 1
                h += (int(r["Q"], 2), int(r["R"], 2)) != (Dv // dv, Dv % dv)
    sonuc["kapsamli"] = (t, h)
    return sonuc


def hepsi():
    return {1: yaprak1(), 2: yaprak2(), 3: yaprak3(), 4: yaprak4()}


if __name__ == "__main__":
    s = hepsi()
    y1 = s[1]
    print("Y1 dört işlem:", [(k["A"], k["B"], k["c0"], k["S"], k["C"], k["uyum"]) for k in y1["islemler"]])
    print("Y1 512 test (toplam, hata):", y1["kapsamli"])
    print("Y1 genel n=1..8:", y1["genel"])
    for r in y1["retler"]:
        print("Y1 ret:", r)
    print("Y1 kayıp:", y1["kayip"])
    y2 = s[2]
    for k in y2["islemler"]:
```

```
    print("Y2", k["M"], "-", k["N"], "borç:", k["borc"][:2], "tüm:", k["tum"][:2], "ref:", k["ref"])
print("Y2 256 test (test, hata_borç, hata_tüm, ilk hata):", y2["kapsamli"])
print("Y2 iddia yanlış durumlar:", y2["iddia_yanlis"])
print("Y2 2tüm(0000):", y2["sifir_tumleyen"], "doğru yöntem N=0 son elde:", y2["dogru_n0"])
y3 = s[3]
for t in y3["tablolar"]:
    print("Y3", t["A"], "x", t["B"], "=", t["P"], int(t["P"], 2), "ref", t["ref"], t["katkilar"])
for t in y3["testler"]:
    print("Y3 test", t)
print("Y3 kapsamlı:", y3["kapsamli"])
y4 = s[4]
for r in y4["islemler"]:
    print("Y4", r["D"], "/", r["d"], "Q", r["Q"], "R", r["R"], "kesir", r["kesir"], "tam", r["tam_mi"],
          "tekrar", r["tekrar"], r["kontrol1"], r["kontrol2"], r["ref"])
for r in y4["sinir"]:
    print("Y4 sınır", r)
for r in y4["hatali"]:
    print("Y4 hatalı", r)
print("Y4 kapsamlı:", y4["kapsamli"])
```